

Assignment: AI Travel Planner — Multi-Agent System

Course: Building AI Agents with LangChain

Topic: Multi-Agent Collaboration with LangGraph

Difficulty: Advanced

Estimated Time: 2–3 weeks

Team Size: Individual or pairs

Background

Real-world AI applications rarely rely on a single agent. Complex tasks — like planning a trip — require multiple specialized agents working together, each with its own tools, context, and responsibilities. This mirrors how real organizations work: a travel agency has separate departments for flights, hotels, itineraries, and budgets.

In this assignment, you will design and build a **multi-agent travel planning system** using **LangGraph** for orchestration and **LangChain** for individual agent logic. By the end, you'll have a working system that takes a user's travel request and produces a complete, coherent travel plan — collaboratively assembled by a team of AI agents.

Learning Objectives

By completing this assignment, you will be able to:

- Design a **multi-agent architecture** with clear role separation
 - Use **LangGraph** to orchestrate agent collaboration and manage state flow
 - Implement **tool use** and bind specific tools to specific agents
 - Share state across agents using a typed state object
 - Apply a **human-in-the-loop** checkpoint for user approval
 - Trace and debug multi-agent runs using **LangSmith**
-

Problem Statement

Build an AI Travel Planner that accepts a natural language travel request from a user, such as:

"Plan a 5-day trip to Tokyo for 2 people in late October. Our budget is \$4,000 USD. We enjoy cultural sites, local food, and some nature."

The system must use multiple collaborating agents to produce a complete travel plan including flights, accommodation, a day-by-day itinerary, and a budget summary.

System Architecture

Your system must include the following **six agents**:

1. Orchestrator Agent

- Parses the user's travel request
- Extracts key parameters: destination, dates, budget, preferences
- Delegates tasks to specialist agents
- Assembles the final plan from all agent outputs

2. Flight Agent

- Searches for available flight options
- Returns at least 2–3 options with price, duration, and airline

- **Tools required:** web search or a mock flight API

3. 🏨 Hotel Agent

- Finds accommodation options matching budget and preferences
- Returns at least 2–3 options with price per night, rating, and location
- **Tools required:** web search or a mock hotel API

4. 📅 Itinerary Agent

- Builds a day-by-day activity plan based on the destination and preferences
- Ensures activities are realistic (travel time, opening hours)
- **Tools required:** web search

5. 💰 Budget Agent

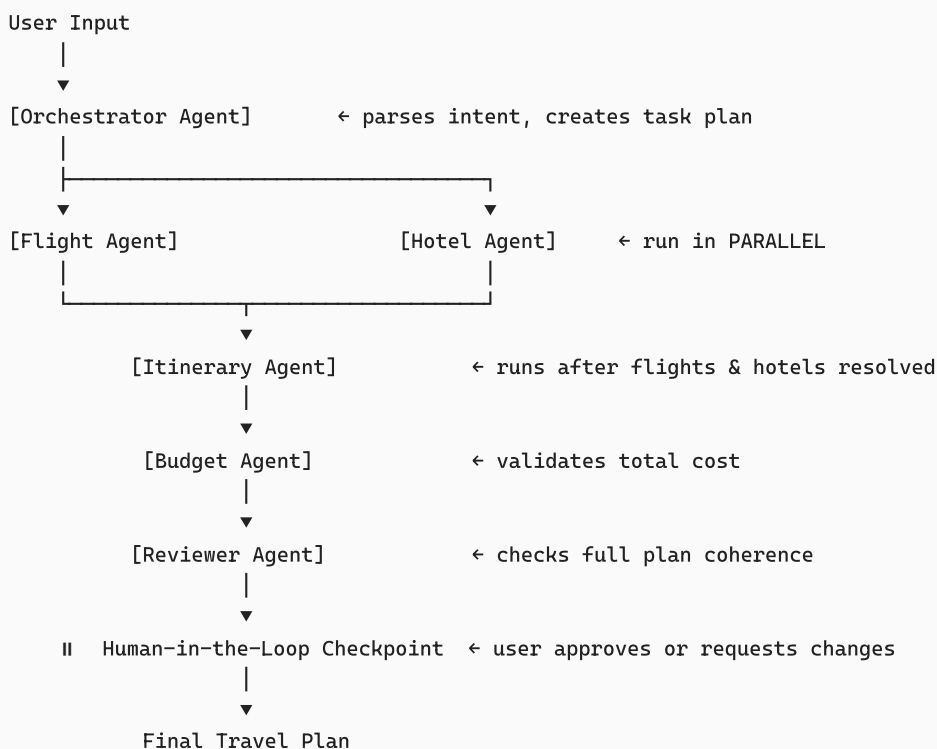
- Aggregates costs from Flight, Hotel, and Itinerary agents
- Validates that the total is within the user's budget
- Flags overruns and suggests trade-offs if needed
- **Tools required:** calculator tool

6. 🔍 Reviewer Agent

- Reviews the complete plan for coherence, gaps, and conflicts
- Checks that dates align, locations are consistent, and the plan is realistic
- Produces a short review note before the plan is shown to the user

🔄 Required Workflow

Your LangGraph workflow must follow this structure:



Note: Flight Agent and Hotel Agent must run in **parallel** using LangGraph's parallel node execution.

🧩 Technical Requirements

Shared State

All agents must read from and write to a single shared state object. Define it using `TypedDict` :

```
from typing import TypedDict, List, Optional

class TravelPlanState(TypedDict):
    # User inputs
    destination: str
    travel_dates: dict          # {"start": "2024-10-22", "end": "2024-10-27"}
    num_travelers: int
    budget_usd: float
    preferences: List[str]

    # Agent outputs
    flights: List[dict]         # list of flight options
    hotels: List[dict]          # list of hotel options
    selected_flight: dict
    selected_hotel: dict
    itinerary: List[dict]       # list of daily plans
    total_cost: float
    budget_status: str          # "within_budget" | "over_budget"
    review_notes: str

    # Control
    approved: bool
    revision_request: Optional[str]
```

Tool Binding

Each agent must only have access to tools relevant to its role:

Agent	Allowed Tools
Orchestrator	None (reasoning only)
Flight Agent	web_search , mock_flight_search
Hotel Agent	web_search , mock_hotel_search
Itinerary Agent	web_search
Budget Agent	calculator
Reviewer Agent	None (reasoning only)

Human-in-the-Loop

Before producing the final output, your graph must **pause** and present the plan to the user. The user can either:

- ✔ **Approve** → finalize and output the plan
- 🔄 **Request changes** → route back to the relevant agent(s) for revision

Project Structure

Organize your project as follows:

```
travel_planner/
├── agents/
│   ├── orchestrator.py
│   ├── flight_agent.py
│   ├── hotel_agent.py
│   ├── itinerary_agent.py
│   ├── budget_agent.py
│   └── reviewer_agent.py
└── tools/
```

```
| | search_tools.py
| | mock_flight_api.py
| | mock_hotel_api.py
| | calculator.py
| graph/
| | state.py          ← TravelPlanState definition
| | workflow.py       ← LangGraph graph definition
| app.py              ← Entry point (Streamlit or CLI)
| .env                ← API keys (do NOT commit)
| README.md
```

✓ Deliverables

#	Deliverable	Description
1	Source Code	Full project on GitHub with clear commit history
2	Architecture Diagram	Visual diagram of your agent workflow (draw.io, Mermaid, etc.)
3	README.md	Setup instructions, how to run, and a sample output
4	LangSmith Trace	Screenshot or exported trace of a complete run
5	Reflection Report	1–2 page report (see prompt below)

Reflection Report Prompts

Answer the following in your report:

1. Why did you assign each tool to its specific agent? What would go wrong if you gave all tools to one agent?
2. What challenges did you face implementing parallel agent execution? How did you solve them?
3. Describe a case where the Reviewer Agent caught a problem. How did your system handle it?
4. What would you add or change if you had more time?

📊 Grading Rubric

Criterion	Weight	Description
Agent Role Design	20%	Each agent has a clear, well-scoped role with appropriate tools
LangGraph Workflow	25%	Correct graph structure including parallel execution and human-in-the-loop
Shared State Management	15%	State is correctly typed, passed, and updated across all agents
Tool Integration	15%	At least 3 tools are implemented and bound to the correct agents
Output Quality	10%	Final travel plan is coherent, complete, and realistic
LangSmith Tracing	5%	A full run is traced and submitted
Reflection Report	10%	Demonstrates understanding of multi-agent design decisions

Total: 100 points

💡 Hints & Tips

- **Start with mocks.** Build each agent in isolation using hardcoded/mock tool responses before wiring them into the graph. This isolates bugs early.
- **State is your contract.** Agree on the shape of `TravelPlanState` before writing any agent logic — changing it later breaks everything.
- **LangSmith is your best friend.** Enable tracing from day one. It will save you hours of print-statement debugging.
- **Test the failure path.** What happens if the Hotel Agent returns no results? Your system should handle it gracefully, not crash.

- **Parallel ≠ independent.** Flight and Hotel agents run at the same time, but the Itinerary Agent depends on both. Make sure your graph enforces this dependency.

🚫 Common Mistakes to Avoid

- Giving all tools to the Orchestrator — this defeats the purpose of multi-agent design
- Hardcoding the selected flight/hotel instead of letting the agent decide
- Skipping the human-in-the-loop checkpoint
- Using `print()` for debugging instead of LangSmith traces
- Committing your `.env` file with API keys

📦 Suggested Tech Stack

Component	Recommended Option
Agent Orchestration	LangGraph
LLM	OpenAI GPT-4o or Anthropic Claude
Web Search Tool	Tavily API or SerpAPI
Observability	LangSmith
UI	Streamlit (optional but encouraged)
Language	Python 3.11+

📅 Suggested Timeline

Week	Milestone
Week 1	M1: Build each agent in isolation with mocked tools. M2: Define shared state and connect agents in LangGraph.
Week 2	M3: Replace mocks with real tools. M4: Implement parallel execution for Flight + Hotel agents.
Week 3	M5: Add human-in-the-loop checkpoint. M6: Trace full run in LangSmith. Write reflection report.

📚 Resources

- [LangGraph Documentation](#)
- [LangGraph Multi-Agent Tutorial](#)
- [LangSmith Tracing Guide](#)
- [Tavily Search API](#)
- [LangChain Tool Binding](#)

Good luck, and enjoy building your first multi-agent system! If your agents start arguing with each other, that's probably a feature, not a bug. 🗑️🤖